Signal-to-Noise: Using Rework Propagation as a Trigger for Dynamic Media Switching

1. Hypotheses and Research Foundations

We propose that team performance in serial engineering tasks is not optimized by a static choice of tools (e.g., "always use Zoom" vs. "always use Jira"), but by a dynamic protocol based on system feedback.

- **H1 (The Indicator):** High rates of **Propagating Rework** (downstream errors caused by upstream changes) are a quantifiable signal of "Dependency Neglect", where the asynchronous information supply was insufficient for the task's complexity.
- **H2 (The Intervention):** Teams utilizing a **"Rework Trigger" protocol**, defaulting to asynchronous tools but switching to synchronous collaboration *only* when rework is detected, will achieve the same quality as fully synchronous teams but with significantly higher efficiency (lower total coordination cost).

Relation to Research Literature

This experiment is grounded in classical coordination theory but extends it using contemporary research on distributed engineering and "Organizational Debt" (see Literature Review section for a complete list).

i. Socio-Technical Congruence (STC) & The "Congruence Gap" The foundational logic of this experiment relies on **Cataldo et al.'s (2006, 2008)** theory of *Socio-Technical Congruence*, which posits that team communication patterns must match the technical dependencies of the code they write.

- **Current Research:** A **2021 Systematic Literature Review** of STC confirmed that "congruence gaps" (where technical dependencies exist without corresponding social interaction) are the primary predictor of build failures in distributed teams.
- **Relevance:** Our experiment directly tests this by using "Rework" as a real-time sensor of this congruence gap.

ii. Organizational Debt as a Driver of Rework While traditional literature views rework merely as "waste," recent research (2023-2024) frames it as "Organizational Debt", the accumulation of outdated structures or siloed processes.

- **Current Research:** A 2024 study on Organizational Debt identifies "siloed efforts" and "poorly managed change" as key drivers that hinder adaptability. Further research indicates that late-stage design changes (rework) are often due to "conflicting perceptions" in global teams, not just technical errors.
- **Relevance:** This supports our H1 hypothesis that rework is a signal of *organizational* misalignment, not just individual coder error.

iii. Remote Work & Dynamic Media Switching Post-pandemic research has challenged the "richer is better" assumption of early Media Richness Theory.

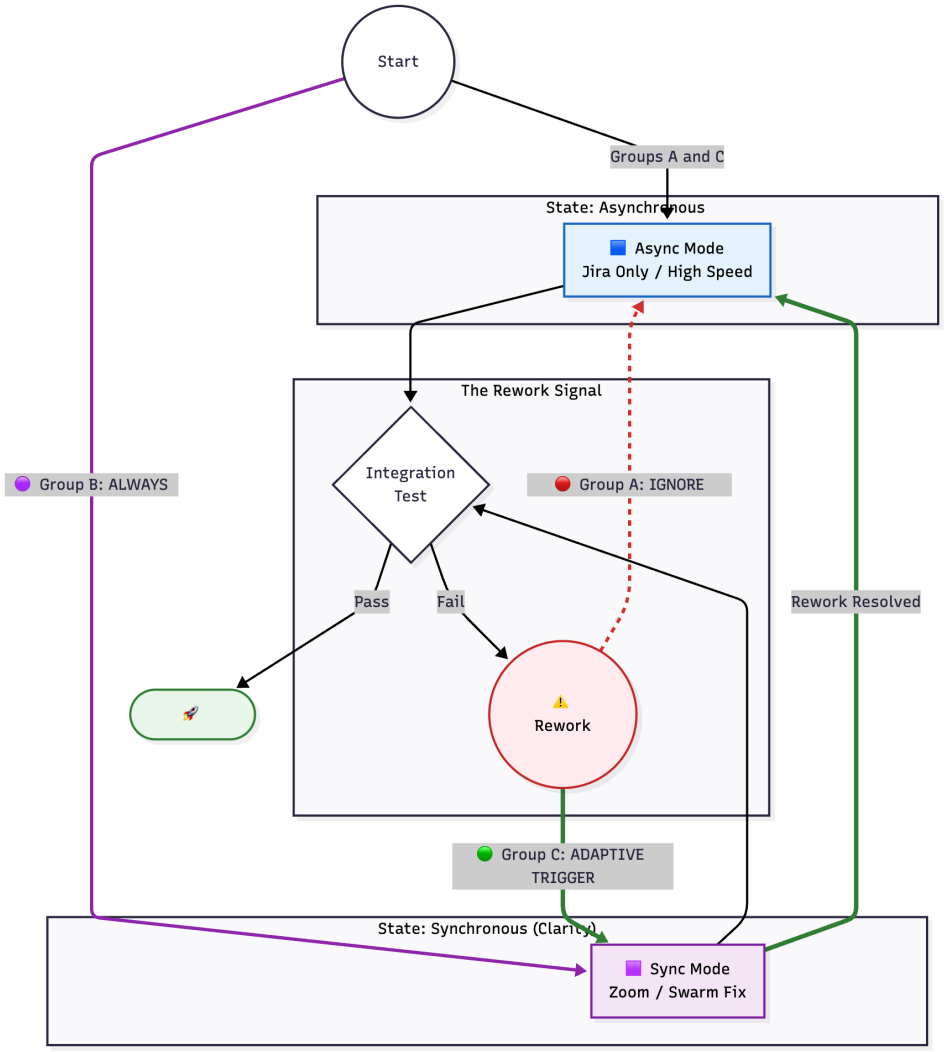
- **Current Research:** Studies from 2022 to 2024 on Agile remote teams found that successful teams do not simply "talk more" (synchronously); they master the art of "Adapting Communication Methods"—shifting to asynchronous documentation to reduce fatigue, but reverting to synchronous channels (video) for complex problem-solving.
- **The Gap:** While current literature observes *that* teams switch methods, it lacks empirical rules for *when* to switch. This experiment fills that gap by testing "Ticket Bounces" as the precise trigger for that switch.

2. The Team Challenge

Teams will undertake a simulated **Collaborative Engineering Task** lasting between 60 and 90 minutes. The task involves a serial workflow (A → B → C) such as a multi-stage system integration or a complex debugging chain where downstream modules depend on upstream logic.

Real-World Context: This setup reflects common engineering scenarios, such as "breaking the build" or failing code review. In the field, these teams often face a dilemma: "Siloed" teams generate code quickly but fail to integrate it (resulting in high rework). In contrast, "Over-collaborative" teams suffer from meeting fatigue and low output volume. This challenge replicates tension to identify communication-related bottlenecks.

3. Experimental Design



We will use a **Between-Subjects Factorial Design**. There is no crossover component; each team experiences only one condition to prevent learning effects (participants getting better at the specific coding task) from confounding the data.

Conditions (Treatments) We compare three protocols for managing coordination supply vs. demand:

- **Group A (Under-Supply / Control): Asynchronous Only.** Teams must use Jira/Text for all steps; no voice/video allowed.
- **Group B (Over-Supply / Control): Synchronous Default.** Teams must keep an "Always-on" Zoom link open.
- **Group C (Adaptive / Treatment): The Rework Trigger.** Teams default to Jira. However, if a task fails validation (requiring rework), they are *required* to open a "Swarm" voice channel to resolve it before returning to asynchronous work.

Independent Variables	Communication Channel	Text	Jira	Zoom	
	Voice Communication	Yes	No		
	t_{sync} Duration of synchronous Event (min)	< 1 min	< 2 min	< 3min	5 min or greater
	Task Score	0	40 (Pass Unit Tests)	80 (Unit Test + Integration Integrity)	100 (Unit Test + Integration Integrity + Specification Adherence)
	M_{async} Asynchronous Message Count	0	1	2	More than 2
Dependent Variables	Rework (Ticket Bounce Rate)	0	1	2	More than 2
	Efficiency Ratio	0 (Poor)	0.25 (Mediocre)	0.5 (acceptable)	0.8+ (superb)

Cognitive Load (λ) is a constant (0.5 min)

4. Relevant Population and Allocation

Participants will be selected from populations representing engineering project teams, including graduate students, early-career engineers, and individuals with experience in software projects. Participants must have baseline familiarity with the tooling (Slack/Jira).

Recruitment and Significance Risks

- **Sample Size:** We aim for 6–8 teams per condition to support statistical power and minimize random variation.
- **Risk (Skill Heterogeneity):** A major risk to significance is that one "10x engineer" could skew a team's results regardless of the communication method.
- **Mitigation:** We will employ a pre-experiment survey to assess coding proficiency. Participants will be stratified and allocated to ensure an equal distribution of skill levels across the three groups.

5. Observation, Sensors, and Sampling

The experiment replaces physical sensors with Digital Telemetry (API Loggers). We will capture timestamped logs from Jira, Slack, and Zoom.

- **Primary Metric (The Trigger):** Ticket Bounce Rate. We explicitly measure the count of times a ticket moves backward in the workflow (e.g., "Testing", "In Progress").

$$B_{total} = \sum_{i=1}^{T} (k_i)$$

where:

- B_{total} : total number of backward movements (bounces)
- k_i : number of backward movements for ticket i
- T : total number of tickets processed

- **Efficiency Ratio:**

$$\eta = \frac{S_{task}}{C_{load}}$$

where:

$$S_{task} = \alpha(T_{unit}) + \beta(I_{int}) + \gamma(P_{spec})$$

$$C_{load} = \sum_{j=1}^{M} (t_{sync}) + \sum_{k=1}^{N} (\lambda \cdot m_{async})$$

Task Score (S_{task}) represents the quality and correctness of the team's engineering output. It is computed from three components:

- T_{unit} : Unit Test Pass Rate (40%) - Percentage of automated tests the team's code passes.
- I_{int} : Integration Integrity (40%) - Whether the team's work integrates without breaking downstream dependencies.
- P_{spec} : Specification Adherence (20%) - How well the delivered solution matches the stated requirements.

We calculate the "tax" paid in communication. Since the experiment uses different channels, we normalize them into Total Coordination Minutes (C_{load})

- t_{sync} : Duration of distinct synchronous events (Zoom/Huddle logs) in minutes.
- m_{async} : Count of asynchronous messages (Slack/Jira comments).
- λ : The Cognitive Load Coefficient (estimated average time to read/context-switch per message, e.g., 0.5 minutes).

Indirect/Qualitative Techniques

- Post-Task Survey: Administered immediately after the session to measure perceived synergy and clarity while memory is fresh.
- Ethnographic Observation: Blinded observers will note behavioral indicators of frustration, hesitation, or "coordination neglect" (ignoring a message).

For data processing, raw logs will be synchronized to create a master timeline. This allows us to correlate specific communication bursts (e.g., a Zoom call in Group C) with the resolution of specific rework loops.

6. <u>Analysis and Bias Reduction</u>

Statistical analysis of the data will be performed using a One-Way Analysis of Variation (ANOVA) to compare the three groups.

- Expected Outcome*:* We expect Group C (Adaptive) to show a statistically significant "sweet spot", lower cycle time than Group B, and lower rework rates than Group A.
- We will specifically test the correlation between Ticket Bounces and Message Volume, testing the hypothesis that rework is a valid signal for increased communication demand.

To reduce bias:

- Observer Bias: Observers will be blinded to the specific hypothesis (i.e., they will not know if they are observing a "Treatment" or "Control" group) and will use standardized scoring rubrics.
- Selection Bias: Minimized through random assignment and skill stratification.
- Procedural Bias: Instructions and task inputs will be identical across all teams to ensure observed effects are due solely to the communication protocol. To reduce skill heterogeneity, participants will be provided with instructions and offline training to get proficient prior to the experiment.
- Authors' Bias: Include and review of counter-argument literature. These papers will present an opposition to our claims, and we will highlight and compare their findings against our hoping to remove the bias and increase the overall validity of the experiment.
    - Webb M. (2021). Zoom Fatigue and How to Prevent It. Journal of registry management, 48(4), 181–182.
    - IxDF - Interaction Design Foundation. (2016, June 5). What is Cognitive Load?. IxDF - Interaction Design Foundation. https://www.interaction-design.org/literature/topics/cognitive-load

- Agrawal, A., Atiq, Mohd. A., & Maurya, L. S. (2016). A Current Study on the Limitations of Agile Methods in Industry Using Secure Google Forms. Procedia Computer Science, 78, 291–297. sciencedirect. https://doi.org/10.1016/j.procs.2016.02.056
- Montrief, T., Haas, M. R. C., Alvarez, A., Gottlieb, M., Siegal, D., & Chan, T. (2020). Thinking Outside the Inbox: Use of Slack in Clinical Groups as a Collaborative Team Communication Platform. AEM education and training, 5(1), 121–129. https://doi.org/10.1002/aet2.10497

References

Agile Business Consortium. (2022). *Remote working in an agile team* [White paper]. https://www.agilebusiness.org

Ahmad, M. O., Lakanen, M., & Kuvaja, P. (2021). Assessing the impact of socio-technical congruence in software development: A systematic literature review. *Kuwait Journal of Science*, *48*(3), 1–15.

Battisti, S., Alfaro, E., & Leonardi, P. (2022). Agility in adversity: Navigating the shift to remote work during the COVID-19 pandemic. *International Journal of Research in Marketing*, *39*(2), 110–128.

Cataldo, M., & Herbsleb, J. D. (2013). Coordination breakdowns and their impact on development productivity and software failures. *IEEE Transactions on Software Engineering*, *39*(3), 343–360. https://doi.org/10.1109/TSE.2012.86

Cataldo, M., Wagstrom, P. A., Herbsleb, J. D., & Carley, K. M. (2006). Identification of coordination requirements: Implications for the design of collaboration and awareness tools. *Proceedings of the 2006 20th Anniversary Conference on Computer Supported Cooperative Work* (pp. 353–362). ACM.

Daft, R. L., & Lengel, R. H. (1986). Organizational information requirements, media richness and structural design. *Management Science*, *32*(5), 554–571. https://doi.org/10.1287/mnsc.32.5.554

Dennis, A. R., Fuller, R. M., & Valacich, J. S. (2008). Media, tasks, and communication processes: A theory of media synchronicity. *MIS Quarterly*, *32*(3), 575–600.

Hameed, A., Waseem, M., & Liang, P. (2021). Exploring the aspects of rework in global software development: An empirical study. *International Journal of Computer Applications*, *174*(13), 15–22.

Malone, T. W., & Crowston, K. (1994). The interdisciplinary study of coordination. *ACM Computing Surveys*, *26*(1), 87–119. https://doi.org/10.1145/174666.174668

Reiman, T., Kaasinen, E., & Väyrynen, S. (2024). Organizational debt—Roadblock to agility in software engineering. *Software Quality Journal*, *32*, 45–68.

Waseem, M., & Liang, P. (2021). Socio-technical congruence as an emerging concept in software development: A scientometric analysis. *IEEE Access*, *9*, 16764–16779. https://doi.org/10.1109/ACCESS.2021.3053455

Weick, K. E., & Sutcliffe, K. M. (2001). *Managing the unexpected: Assuring high performance in an age of complexity*. Jossey-Bass.